

run_bank

run_bank.sh — Helix OTA HelixQA challenge-bank runner

Revision: 1 Last modified: 2026-06-10T11:49:44Z

Overview

tools/helixqa/run_bank.sh machine-executes the HelixQA challenge bank at tools/helixqa/banks/helix_ota.yaml so the bank is no longer a declared-only manifest that a human must hand-run (the §11.4.27 incorporation gap). For each challenge it enforces the same two anti-bluff gates the canonical HelixQA engine (pkg/testbank/dispatch.go in HelixDevelopment/helixqa) enforces:

1. **Dispatch-exit gate** — the challenge's dispatch_command MUST exit 0 (exit 3 = honest SKIP-with-reason per §11.4.3; any other non-zero = FAIL).
2. **Evidence-ledger gate (§11.4.69)** — every declared evidence_artifact MUST resolve to a real, NON-EMPTY file. A zero-exit dispatch with a missing/empty artefact FAILs — a green command never excuses absent evidence.

Prerequisites

- bash, awk (runner itself).
- For a LIVE run: the dispatched scripts need curl, jq, go, openssl etc., and the shared-server challenges need HELIX_ADMIN_PASSWORD set; the self-hosting challenges mint their own ephemeral server + key.

Usage

```
# Static audit – verify every challenge points to a real dispatch
#                      command AND
# its evidence artefact is present + non-empty. Touches nothing
#                      live.
bash tools/helixqa/run_bank.sh --dry-run

# §1.1 paired-mutation self-test – proves the runner's evidence
#                      ledger catches
```

```
# its own negation (a bluff bank with absent evidence MUST FAIL; a
    real-evidence
# bank MUST PASS). Self-cleaning, deterministic.
bash tools/helixqa/run_bank.sh --self-test

# LIVE full-bank run – runs every dispatch_command against the
    real system and
# scores PASS only on captured evidence.
HELIX_ADMIN_PASSWORD=<pw> bash tools/helixqa/run_bank.sh

# Custom bank
bash tools/helixqa/run_bank.sh --bank <path>
```

Exit 0 only if every non-skipped challenge PASSEd.

Edge cases

- A challenge with NO dispatch_command is a declared-only bluff → --dry-run FAILs it.
- A challenge whose evidence_artifact is missing or 0-byte → FAIL (the §11.4.69 hole the gate closes).
- Missing prerequisite in a LIVE dispatch → the dispatched script exits 3 → the runner reports SKIP, never a false PASS.

Internal behaviour

parse_bank reads the YAML with awk (no yaml dependency), emitting <challenge_id>\t<dispatch_command>\t<evidence_artifact> per challenge, handling folded (>) continuation lines. evidence_ok resolves relative tokens against the repo root, absolute tokens as-is, and requires the file to exist and be non-empty.

Related scripts

- tests/e2e/challenge_operational.sh, tests/e2e/pipeline_signed.sh, tests/e2e/recall_lifecycle.sh, tests/e2e/challenge_filters_pagination.sh, tests/security/security_probes.sh — the real anti-bluff challenge bodies the bank dispatches to.

Last verified

2026-06-10 — --dry-run 10/0/0 PASS; --self-test SELF-TEST PASS (3/3 deterministic); negative-control mutated bank → RESULT: FAIL.